```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt
```

Q1: Basics of CNNs – CIFAR-10 Classification

```
# Loading the CIFAR-10 dataset from keras.datasets.
print("\nLoading CIFAR-10 dataset...")
(x_train, y_train_labels), (x_test, y_test_labels) = cifar10.load_data()
print("CIFAR-10 dataset loaded.")
```

```
Loading CIFAR-10 dataset...
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 150s 1us/step
CIFAR-10 dataset loaded.
```

```
# Preprocessing the data
# Normalizing pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

```
# One hot encode the labels
num_classes = 10
y_train = to_categorical(y_train_labels, num_classes)
y_test = to_categorical(y_test_labels, num_classes)
```

```
# 2) Build a CNN with the following:2 convolutional layers, MaxPooling,  Flatten, Dense, Softmax
print("\nBuilding the CNN model...")
model = Sequential(name="Basic_CIFAR10_CNN")
```

```
Building the CNN model...
```

```
# Convolutional Layer 1
model.add(Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)))
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, pref
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# MaxPooling Layer 1
model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
# Convolutional Layer 2
model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
```

```
# Flattening the 2D arrays for fully connected layers
model.add(Flatten())
```

```
# Dense Layer
model.add(Dense(128, activation='relu'))
```

```
# Output Layer
model.add(Dense(num_classes, activation='softmax'))
```

```
# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy', # For one hot encoded labels
              metrics=['accuracy'])
```

```
print("Model built and compiled.")
model.summary()
```

```
Model built and compiled.
Model: "Basic_CIFAR10_CNN"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
flatten (Flatten)	(None, 16384)	0
dense (Dense)	(None, 128)	2,097,280
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 2,117,962 (8.08 MB)
Trainable params: 2,117,962 (8.08 MB)
Non-trainable params: 0 (0.00 B)
```

```
# Training the model for at least 10 epochs.
num_epochs_q1 = 10
batch_size_q1 = 64
```

```
print(f"\n Starting training for {num_epochs_q1} epochs...")
history = model.fit(x_train, y_train,
                    epochs=num_epochs_q1,
                    batch_size=batch_size_q1,
                    validation_data=(x_test, y_test), # Using test set for validation
                    verbose=1)
print("Training complete.")
```

```
 Starting training for 10 epochs...
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 10s 8ms/step - accuracy: 0.4156 - loss: 1.6189 - val_accuracy: 0.6040 - val_loss: 1.1070
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.6352 - loss: 1.0345 - val_accuracy: 0.6520 - val_loss: 0.9909
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.6984 - loss: 0.8580 - val_accuracy: 0.6688 - val_loss: 0.9695
Epoch 4/10
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.7456 - loss: 0.7315 - val_accuracy: 0.6819 - val_loss: 0.9107
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.7886 - loss: 0.6100 - val_accuracy: 0.6871 - val_loss: 0.9406
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.8288 - loss: 0.4926 - val_accuracy: 0.6914 - val_loss: 0.9467
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8685 - loss: 0.3841 - val_accuracy: 0.6992 - val_loss: 0.9961
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8995 - loss: 0.2979 - val_accuracy: 0.6880 - val_loss: 1.1280
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9314 - loss: 0.2114 - val_accuracy: 0.6823 - val_loss: 1.2313
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9506 - loss: 0.1533 - val_accuracy: 0.6789 - val_loss: 1.4410
Training complete.
```

```
# Plotting training and validation accuracy/loss.
print("\nPlotting training and validation accuracy/loss...")
plt.figure(figsize=(12, 5))
```

```
# Plotting training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.grid(True)

# Plotting training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper right')
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
# Printing classification accuracy on test set.
print("\n Evaluating model on the test set...")
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy*100:.2f}%")
```

```
 Evaluating model on the test set...
Test Loss: 1.4410
Test Accuracy: 67.89%
```

Q2: Transfer Learning with Pre-trained CNNs

```
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.applications import ResNet50
```

```
# 1) Load the CIFAR-10 dataset.
# (This was also done in Q1, ensuring data is ready)
print("\nLoading and preprocessing CIFAR-10 dataset...")
(x_train, y_train_labels), (x_test, y_test_labels) = cifar10.load_data()
```

```
Loading and preprocessing CIFAR-10 dataset...
```

```
# Normalizing pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

```
# One hot encode the labels
num_classes = 10
y_train = to_categorical(y_train_labels, num_classes)
y_test = to_categorical(y_test_labels, num_classes)
```

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer','dog', 'frog', 'horse', 'ship', 'truck']
```

```
# 2) Use a pre-trained model (ResNet50) from keras.applications with:
#    include_top=False, input shape matching CIFAR-10
print("\nLoading pre-trained ResNet50 base model...")
base_model_resnet = ResNet50(weights='imagenet',include_top=False, input_shape=(32, 32, 3))
print("ResNet50 base model loaded.")
```

```
Loading pre-trained ResNet50 base model...
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
ResNet50 base model loaded.
```

```
# Freezing all base layers.
print("Freezing layers of the ResNet50 base model...")
base_model_resnet.trainable = False # Freezes all layers in the base model
print("Base model layers frozen.")
```

```
Freezing layers of the ResNet50 base model...
Base model layers frozen.
```

```
# Adding custom GlobalAveragePooling2D -> Dense -> Softmax head.
print("\n Adding custom classification head...")
inputs = base_model_resnet.input
x = base_model_resnet.output    # Output of the base model
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x) # A new dense hidden layer
outputs = Dense(num_classes, activation='softmax')(x) # Final output layer
```

```
 Adding custom classification head...
```

```
# Creating the new model
model_transfer = Model(inputs=inputs, outputs=outputs, name="ResNet50_Transfer_CIFAR10")
print("Custom head added. New model created.")
```

```
Custom head added. New model created.
```

```
# Training only the top layers.
print("\nCompiling the transfer learning model...")
model_transfer.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                       loss='categorical_crossentropy',
                       metrics=['accuracy'])
print("Model compiled.")
```

```
Compiling the transfer learning model...
Model compiled.
```

```
num_epochs_q2 = 10
batch_size_q2 = 64
```

```
print(f"\nStarting training for {num_epochs_q2} epochs (only top layers)...")
history_transfer = model_transfer.fit(x_train, y_train,
                                      epochs=num_epochs_q2,
                                      batch_size=batch_size_q2,
                                      validation_data=(x_test, y_test),
                                      verbose=1)
print("Training of top layers complete.")
```

```
Starting training for 10 epochs (only top layers)...
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 29s 23ms/step - accuracy: 0.1999 - loss: 2.1777 - val_accuracy: 0.3030 - val_loss: 1.9170
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 28s 11ms/step - accuracy: 0.3046 - loss: 1.9151 - val_accuracy: 0.3295 - val_loss: 1.8513
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 11s 13ms/step - accuracy: 0.3360 - loss: 1.8450 - val_accuracy: 0.3410 - val_loss: 1.8407
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 8s 11ms/step - accuracy: 0.3485 - loss: 1.8006 - val_accuracy: 0.3586 - val_loss: 1.8014
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.3573 - loss: 1.7894 - val_accuracy: 0.3603 - val_loss: 1.7762
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.3681 - loss: 1.7486 - val_accuracy: 0.3797 - val_loss: 1.7279
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 10s 13ms/step - accuracy: 0.3715 - loss: 1.7396 - val_accuracy: 0.3691 - val_loss: 1.7701
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.3845 - loss: 1.7186 - val_accuracy: 0.3965 - val_loss: 1.6817
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 10s 13ms/step - accuracy: 0.3879 - loss: 1.7041 - val_accuracy: 0.3856 - val_loss: 1.7197
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 8s 11ms/step - accuracy: 0.3929 - loss: 1.6896 - val_accuracy: 0.3970 - val_loss: 1.6910
Training of top layers complete.
```

```
# Reporting accuracy and showing predictions on 5 misclassified images.
print("\nEvaluating model on the test set...")
test_loss_transfer, test_accuracy_transfer = model_transfer.evaluate(x_test, y_test, verbose=0)
print(f"Test Loss (Transfer Learning): {test_loss_transfer:.4f}")
print(f"Test Accuracy (Transfer Learning): {test_accuracy_transfer*100:.2f}%")
```

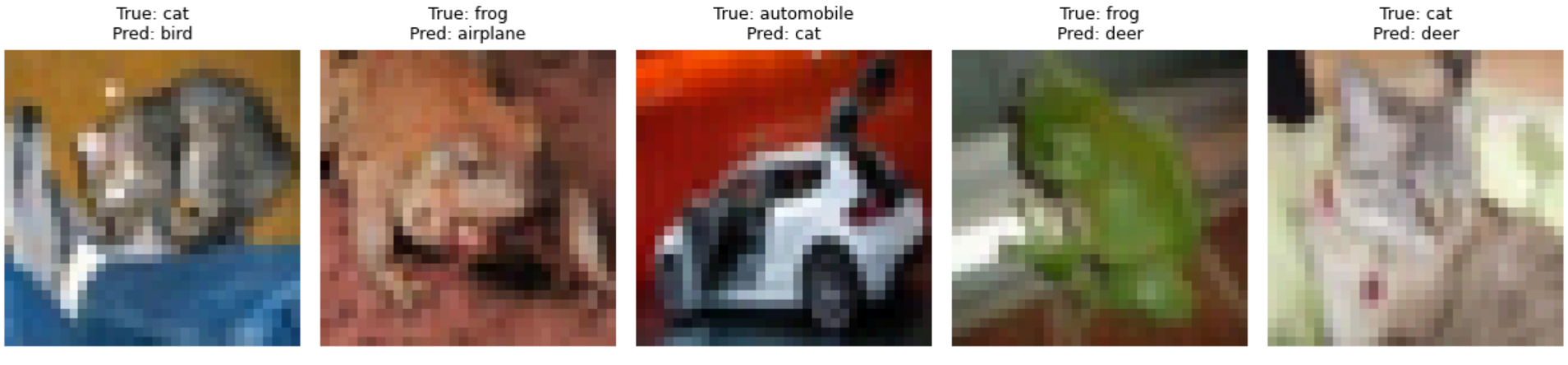
```
Evaluating model on the test set...
Test Loss (Transfer Learning): 1.6910
Test Accuracy (Transfer Learning): 39.70%
```

```
# Finding and displaying 5 misclassified images
print("\nIdentifying and displaying 5 misclassified images...")
predictions = model_transfer.predict(x_test)
predicted_classes = np.argmax(predictions, axis=1)
true_classes = y_test_labels.flatten() # Original integer labels
```

```
Identifying and displaying 5 misclassified images...
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step
```

```
misclassified_indices = np.where(predicted_classes != true_classes)[0]
num_misclassified_to_show = 5
```

```
plt.figure(figsize=(12, 3 * ((min(num_misclassified_to_show, len(misclassified_indices)) + 2) // 3)))
plt.suptitle("Misclassified Images", fontsize=14)
for i, mis_idx in enumerate(misclassified_indices[:num_misclassified_to_show]):
    plt.subplot(1, num_misclassified_to_show, i + 1) # Display in a single row
    plt.imshow(x_test[mis_idx])
    plt.title(f"True: {class_names[true_classes[mis_idx]]}\nPred: {class_names[predicted_classes[mis_idx]]}", fontsize=9)
    plt.axis('off')
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



Start coding or generate with AI.

Q3: RNNs – Character-Level Text Generation

```
from tensorflow.keras.layers import LSTM, Embedding
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.sequence import pad_sequences
import random
```

```
# 1) Use a small plain text file (e.g., a few paragraphs of English text).
# For simplicity, we'll define a small text corpus directly in the code.
text_corpus = """
Alice was beginning to get very tired of sitting by her sister on the bank,
and of having nothing to do: once or twice she had peeped into the
book her sister was reading, but it had no pictures or conversations in
it, 'and what is the use of a book,' thought Alice 'without pictures or
conversation?'

So she was considering in her own mind (as well as she could, for the
hot day made her feel very sleepy and stupid), whether the pleasure
of making a daisy-chain would be worth the trouble of getting up and
picking the daisies, when suddenly a White Rabbit with pink eyes ran
close by her.
"""
```

```
print(f"Using text corpus of length: {len(text_corpus)} characters.")
```

```
Using text corpus of length: 594 characters.
```

```
# 2) Preprocess: Character-level tokenization
print("\nPreprocessing text: Character-level tokenization...")
chars = sorted(list(set(text_corpus)))
vocab_size = len(chars)
print(f"Vocabulary size (unique characters): {vocab_size}")
```

```
Preprocessing text: Character-level tokenization...
Vocabulary size (unique characters): 36
```

```
# Create character to index and index to character mappings
char_to_index = {char: i for i, char in enumerate(chars)}
index_to_char = {i: char for i, char in enumerate(chars)}
```

```
# Convert the entire text to a sequence of integers
encoded_text = [char_to_index[char] for char in text_corpus]
```

```
# Creating sequences of characters and their corresponding next characters
seq_length = 50 # Length of input character sequences
step = 1        # Step to create next sequence
input_sequences = []
output_chars_idx = []
```

```
for i in range(0, len(encoded_text) - seq_length, step):
    input_sequences.append(encoded_text[i : i + seq_length])
    output_chars_idx.append(encoded_text[i + seq_length])

num_sequences = len(input_sequences)
```

```
print(f"Number of sequences created: {num_sequences}")
```

```
Number of sequences created: 544
```

```
# Preparing data for Keras model
X = np.zeros((num_sequences, seq_length), dtype=np.int32)
y = np.zeros((num_sequences, vocab_size), dtype=np.bool_)
```

```
for i, seq in enumerate(input_sequences):
    X[i] = seq
    y[i, output_chars_idx[i]] = 1
```

```
print(f"Shape of X (input sequences): {X.shape}")
print(f"Shape of y (one-hot encoded target characters): {y.shape}")
print("Preprocessing complete.")
```

```
Shape of X (input sequences): (544, 50)
Shape of y (one-hot encoded target characters): (544, 36)
Preprocessing complete.
```

```
# Building a simple LSTM model to predict the next character.
embedding_dim = 64 # Dimension of character
lstm_units = 128   # Number of units in the LSTM layer
```

```
print("\nBuilding the LSTM model...")
model_char_rnn = Sequential(name="CharLSTM_TextGen")
model_char_rnn.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=seq_length))
model_char_rnn.add(LSTM(units=lstm_units))
model_char_rnn.add(Dense(vocab_size, activation='softmax'))
```

```
Building the LSTM model...
```

```
# Compiling the model
model_char_rnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
print("Model built and compiled.")
```

```
Model built and compiled.
```

```
# Training for a few epochs and generating text
num_epochs_q3 = 20
batch_size_q3 = 128
```

```
print(f"\nStarting training for {num_epochs_q3} epochs...")
history_char_rnn = model_char_rnn.fit(X, y, epochs=num_epochs_q3,batch_size=batch_size_q3, verbose=1) # Show progress
print("Training complete.")
```

```
Starting training for 20 epochs...
Epoch 1/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.0824 - loss: 3.5773
Epoch 2/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1630 - loss: 3.5169
Epoch 3/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1580 - loss: 3.1969
Epoch 4/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1738 - loss: 3.0549
Epoch 5/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1530 - loss: 3.0232
Epoch 6/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1728 - loss: 2.9967
Epoch 7/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1796 - loss: 2.9943
Epoch 8/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1725 - loss: 2.9891
Epoch 9/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1724 - loss: 2.9533
Epoch 10/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1694 - loss: 2.9884
Epoch 11/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1554 - loss: 2.9861
Epoch 12/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1751 - loss: 2.9527
Epoch 13/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1794 - loss: 2.9331
Epoch 14/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1576 - loss: 2.9603
Epoch 15/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1878 - loss: 2.9121
Epoch 16/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1771 - loss: 2.8859
Epoch 17/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1871 - loss: 2.8568
Epoch 18/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1821 - loss: 2.9006
Epoch 19/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.1950 - loss: 2.8434
Epoch 20/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.2220 - loss: 2.8424
Training complete.
```

```
# Function to generate text
def generate_text(model, seed_string, char_to_idx, idx_to_char, seq_len, num_chars_to_gen=200):
    print(f"\nGenerating text with seed: \"{seed_string}\"")
    generated_text = seed_string
    # Prepare the initial seed sequence
    current_sequence_str = seed_string[-seq_len:]

    for _ in range(num_chars_to_gen):
        # Convert current sequence to integer indices
        int_sequence = [char_to_idx.get(char, 0) for char in current_sequence_str]

        # Pad if current_sequence_str is shorter than seq_len
        padded_sequence = pad_sequences([int_sequence], maxlen=seq_len, padding='pre', truncating='pre')

        # Predict probabilities for the next character
        predicted_probs = model.predict(padded_sequence, verbose=0)[0]

        # Sample the next character index based on probabilities
        next_char_index = np.random.choice(len(predicted_probs), p=predicted_probs)

        next_char = idx_to_char.get(next_char_index, '?') # Using '?' if index is somehow out of bounds

        generated_text += next_char
        # Updating the current sequence for the next prediction
        current_sequence_str = current_sequence_str[1:] + next_char

    return generated_text
```

```
# Generate text using a seed string
seed_text = "Alice was beginning to get very tired of sitting by her sister"
generated_output = generate_text(model_char_rnn, seed_text, char_to_index, index_to_char, seq_length)
```

```
Generating text with seed: "Alice was beginning to get very tired of sitting by her sister"
```

```
print("\n Generated Text :-")
print(generated_output)
```

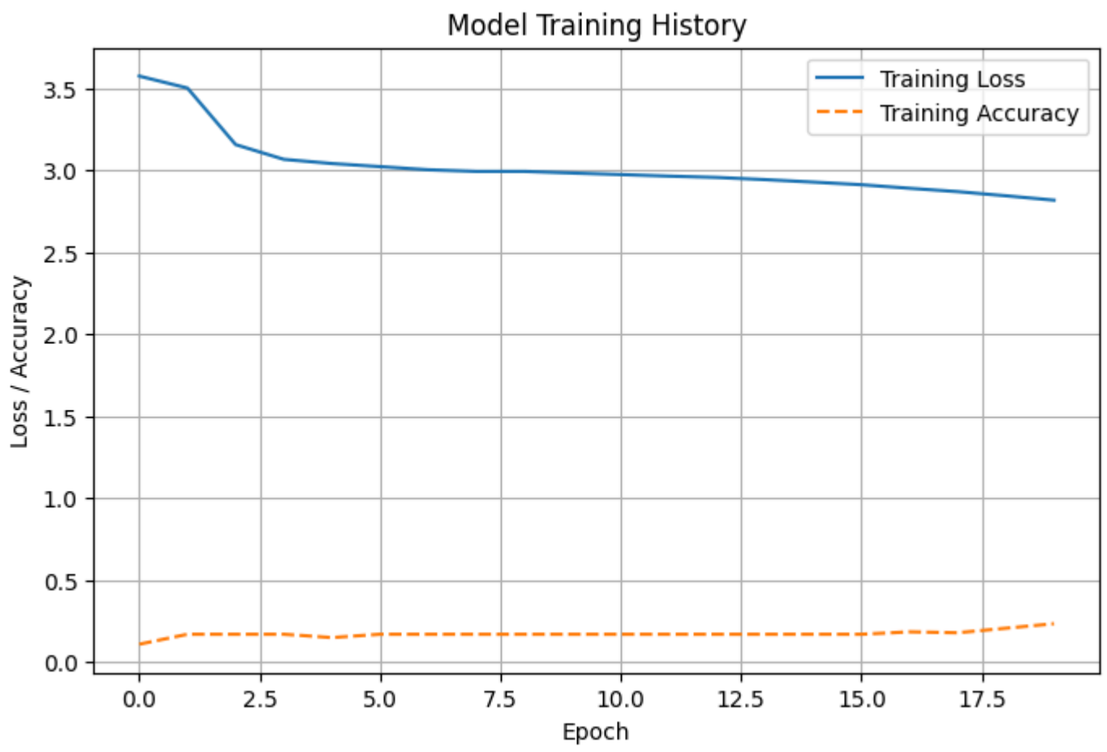
```
 Generated Text :-
Alice was beginning to get very tired of sitting by her sistertg'g oe klrn n rk ch a rhcisy rinrertytho o ,h ceae ngs r nbneillt i,lcs  ha 'ac y) A wdwhgrah the 'g aisl b t ed.t hisaw sansp?n,
lhathbhea waicbsn  h te np
ahierikttpcd ta cuamhe  ehhtrto tnhu owhS
```

the generated text is not fully coherent, which is expected given the model's simplicity and limited training data/epochs, but it demonstrates the character-level generation process.

```
# Plotting training loss.
print("\nPlotting training loss...")
plt.figure(figsize=(8, 5))
```

```
plt.plot(history_char_rnn.history['loss'], label='Training Loss')
if 'accuracy' in history_char_rnn.history:
    plt.plot(history_char_rnn.history['accuracy'], label='Training Accuracy', linestyle='--')
plt.title('Model Training History')
plt.xlabel('Epoch')
plt.ylabel('Loss / Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

```
Plotting training loss...
```



Start coding or generate with AI.

Q4: Transformers – Text Summarization or Translation

```
!pip install transformers torch
import torch
from transformers import AutoModel, AutoTokenizer
from transformers import BartTokenizer, BartForConditionalGeneration
```

```
━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 32.7 MB/s eta 0:00:00
da_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 35.3 MB/s eta 0:00:00
dnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 2.0 MB/s eta 0:00:00
fft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━ 211.5/211.5 MB 1.7 MB/s eta 0:00:00
rand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl (56.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 13.7 MB/s eta 0:00:00
solver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl (127.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 MB 9.4 MB/s eta 0:00:00
sparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl (207.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━ 207.5/207.5 MB 1.6 MB/s eta 0:00:00
jitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (21.1 MB)
━━━━━━━━━━━━━━━━━━━━━━━━ 21.1/21.1 MB 38.8 MB/s eta 0:00:00
packages: nvidia-nvjitlink-cu12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, nvidia-cusparse-cu12, nvidia-cu
l: nvidia-nvjitlink-cu12
stallation: nvidia-nvjitlink-cu12 12.5.82
ia-nvjitlink-cu12-12.5.82:
nstalled nvidia-nvjitlink-cu12-12.5.82
l: nvidia-curand-cu12
stallation: nvidia-curand-cu12 10.3.6.82
ia-curand-cu12-10.3.6.82:
nstalled nvidia-curand-cu12-10.3.6.82
l: nvidia-cufft-cu12
stallation: nvidia-cufft-cu12 11.2.3.61
ia-cufft-cu12-11.2.3.61:
nstalled nvidia-cufft-cu12-11.2.3.61
l: nvidia-cuda-runtime-cu12
stallation: nvidia-cuda-runtime-cu12 12.5.82
ia-cuda-runtime-cu12-12.5.82:
nstalled nvidia-cuda-runtime-cu12-12.5.82
l: nvidia-cuda-nvrtc-cu12
stallation: nvidia-cuda-nvrtc-cu12 12.5.82
ia-cuda-nvrtc-cu12-12.5.82:
nstalled nvidia-cuda-nvrtc-cu12-12.5.82
l: nvidia-cuda-cupti-cu12
stallation: nvidia-cuda-cupti-cu12 12.5.82
ia-cuda-cupti-cu12-12.5.82:
nstalled nvidia-cuda-cupti-cu12-12.5.82
l: nvidia-cublas-cu12
stallation: nvidia-cublas-cu12 12.5.3.2
ia-cublas-cu12-12.5.3.2:
nstalled nvidia-cublas-cu12-12.5.3.2
l: nvidia-cusparse-cu12
stallation: nvidia-cusparse-cu12 12.5.1.3
ia-cusparse-cu12-12.5.1.3:
nstalled nvidia-cusparse-cu12-12.5.1.3
l: nvidia-cudnn-cu12
stallation: nvidia-cudnn-cu12 9.3.0.75
ia-cudnn-cu12-9.3.0.75:
nstalled nvidia-cudnn-cu12-9.3.0.75
l: nvidia-cusolver-cu12
stallation: nvidia-cusolver-cu12 11.6.3.83
ia-cusolver-cu12-11.6.3.83:
nstalled nvidia-cusolver-cu12-11.6.3.83
d nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-cu12-12.4.127 nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3 nvidia-curand-c
```

```
# Loading BART model and tokenizer for custom prompt handling
model_name = "facebook/bart-large-cnn"
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained(model_name)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
vocab.json: 100%  899k/899k [00:00<00:00, 5.46MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 12.2MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 18.3MB/s]
config.json: 100%  1.58k/1.58k [00:00<00:00, 130kB/s]
model.safetensors: 100%  1.63G/1.63G [00:54<00:00, 17.1MB/s]
generation_config.json: 100%  363/363 [00:00<00:00, 26.9kB/s]
```

```
# Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

```
BartForConditionalGeneration(
  (model): BartModel(
    (shared): BartScaledWordEmbedding(50264, 1024, padding_idx=1)
    (encoder): BartEncoder(
      (embed_tokens): BartScaledWordEmbedding(50264, 1024, padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 1024)
      (layers): ModuleList(
        (0-11): 12 x BartEncoderLayer(
          (self_attn): BartSdpaAttention(
            (k_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (v_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (q_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (out_proj): Linear(in_features=1024, out_features=1024, bias=True)
          )
          (self_attn_layer_norm): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
          (activation_fn): GELUActivation()
          (fc1): Linear(in_features=1024, out_features=4096, bias=True)
          (fc2): Linear(in_features=4096, out_features=1024, bias=True)
          (final_layer_norm): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
        )
      )
      (layernorm_embedding): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
    )
    (decoder): BartDecoder(
      (embed_tokens): BartScaledWordEmbedding(50264, 1024, padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 1024)
      (layers): ModuleList(
        (0-11): 12 x BartDecoderLayer(
          (self_attn): BartSdpaAttention(
            (k_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (v_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (q_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (out_proj): Linear(in_features=1024, out_features=1024, bias=True)
          )
          (activation_fn): GELUActivation()
          (self_attn_layer_norm): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
          (encoder_attn): BartSdpaAttention(
            (k_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (v_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (q_proj): Linear(in_features=1024, out_features=1024, bias=True)
            (out_proj): Linear(in_features=1024, out_features=1024, bias=True)
          )
          (encoder_attn_layer_norm): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
          (fc1): Linear(in_features=1024, out_features=4096, bias=True)
          (fc2): Linear(in_features=4096, out_features=1024, bias=True)
          (final_layer_norm): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
        )
      )
      (layernorm_embedding): LayerNorm((1024,), eps=1e-05, elementwise_affine=True)
    )
  )
  (lm_head): Linear(in_features=1024, out_features=50264, bias=False)
)
```

```
print("BART model loaded successfully!")
```

```
BART model loaded successfully!
```

```
# Function to generate summary with custom prompts
def generate_summary_with_prompt(text, prompt_prefix="", max_length=100, min_length=25):
    # Combine prompt with text
    input_text = prompt_prefix + text

    # Tokenize input
    inputs = tokenizer.encode(input_text, return_tensors="pt", max_length=1024, truncation=True)
    inputs = inputs.to(device)

    # Generate summary
    with torch.no_grad():
        summary_ids = model.generate(inputs, max_length=max_length, min_length=min_length,
                                     do_sample=False, early_stopping=True)

    # Decode summary
    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
    return summary
```

```
# Long article for testing
long_article = """
  The field of renewable energy has experienced unprecedented growth over the past decade, driven by technological advances, declining costs,
  and increasing environmental awareness. Solar photovoltaic systems have become the cheapest source of electricity in many regions,
  with installation costs dropping by over 80% since 2010. Wind energy has similarly benefited from larger, more efficient turbines and improved grid integration technologies.
  Governments worldwide are implementing policies to accelerate the transition from fossil fuels, including carbon pricing mechanisms, renewable energy standards,
  and substantial subsidies for clean energy projects. The International Energy Agency projects that renewable sources will account for 90% of the increase in
  global electricity capacity through 2025. However, challenges remain in energy storage, grid stability, and the intermittent nature of solar and wind power.
  Battery technology improvements and smart grid developments are addressing these issues, making renewable energy more reliable and cost-effective.
  The transition to clean energy is creating millions of jobs while reducing greenhouse gas emissions and air pollution in urban areas.
"""
```

```
print("\n EXAMPLE 1: Standard Summarization :- ")
print(f"Original text length: {len(long_article.split())} words")

standard_summary = generate_summary_with_prompt(long_article, max_length=80, min_length=25)
standard_summary_1= standard_summary.replace('. ', '.\n')
print("\nStandard Summary:")
print(standard_summary_1)
```

```
 EXAMPLE 1: Standard Summarization :-
Original text length: 163 words

Standard Summary:
Solar photovoltaic systems have become the cheapest source of electricity in many regions.
Wind energy has similarly benefited from larger, more efficient turbines and improved grid integration technologies.
Governments worldwide are implementing policies to accelerate the transition from fossil fuels.
```

```
business_article = """
E-commerce platforms have fundamentally transformed retail business models, accelerating digital transformation across industries.
Amazon, Alibaba, and other major platforms have created ecosystems that connect millions of sellers with consumers globally.
Small businesses now have access to international markets through these platforms, while consumers benefit from increased choice and competitive pricing.
 The COVID-19 pandemic further accelerated online shopping adoption, with many traditional retailers rapidly developing digital capabilities.
 Mobile commerce has become increasingly important, with smartphones accounting for over half of online transactions in many markets.
 Social commerce, where purchases are made directly through social media platforms, represents the next frontier in digital retail.
 However, challenges include counterfeit products, data privacy concerns, and the environmental impact of increased packaging and shipping.
"""
```

```
business_summary = generate_summary_with_prompt(business_article, max_length=75, min_length=20)
print(f"Original text length: {len(business_article.split())} words")
business_summary_1= business_summary.replace('. ', '.\n')
print("\nBusiness Article Summary:")
print(business_summary_1)
```

```
Original text length: 119 words

Business Article Summary:
Small businesses now have access to international markets through these platforms.
Mobile commerce has become increasingly important, with smartphones accounting for over half of online transactions in many markets.
Social commerce, where purchases are made directly through social media platforms, represents the next frontier in digital retail.
```

```
science_article = """
Gene editing technology, particularly CRISPR-Cas9, has revolutionized biomedical research and therapeutic development.
This precise molecular tool allows scientists to modify DNA sequences in living cells, offering potential treatments for genetic diseases, cancer, and other conditions.
 Recent clinical trials have shown promising results for treating sickle cell disease, beta-thalassemia, and certain types of blindness.
  Agricultural applications include developing crops with improved nutritional content, disease resistance, and climate adaptability.
  However, ethical concerns about human germline editing and potential unintended consequences have led to calls for strict regulatory oversight.
 The technology's accessibility has democratized genetic research, enabling smaller laboratories to conduct sophisticated experiments previously limited to major research institutions.
"""
```

```
science_summary = generate_summary_with_prompt(science_article, max_length=85, min_length=30)
print(f"Original text length: {len(science_article.split())} words")
science_summary_1= science_summary.replace('. ', '.\n')
print("\nScience Article Summary:")
print(science_summary_1)
```

```
Original text length: 106 words

Science Article Summary:
Gene editing technology, particularly CRISPR-Cas9, has revolutionized biomedical research and therapeutic development.
Recent clinical trials have shown promising results for treating sickle cell disease, beta-thalassemia, and certain types of blindness.
ethical concerns about human germline editing and potential unintended consequences have led to calls for strict regulatory oversight.
```

```
print("\nCHANGING PROMPTS AND OBSERVING CHANGES :- ")
```

```
CHANGING PROMPTS AND OBSERVING CHANGES :-
```

```
# Using the renewable energy article
test_article = long_article
```

```
# Prompt A: Standard (no prefix)
print("--- Prompt A: Standard (No Prefix) ---")
prompt_a = ""
summary_a = generate_summary_with_prompt(test_article, prompt_a, max_length=80, min_length=25)
summary_a_1= summary_a.replace('. ', '.\n')
print(f"Prompt: '{prompt_a}' (empty)")
print(f"Summary: {summary_a_1}")
```

```
--- Prompt A: Standard (No Prefix) ---
Prompt: '' (empty)
Summary: Solar photovoltaic systems have become the cheapest source of electricity in many regions.
Wind energy has similarly benefited from larger, more efficient turbines and improved grid integration technologies.
Governments worldwide are implementing policies to accelerate the transition from fossil fuels.
```

```
# Prompt B: Brief overview request
print("\n Prompt B: Brief Overview Request ")
prompt_b = "Provide a brief overview of the following text: "
summary_b = generate_summary_with_prompt(test_article, prompt_b, max_length=80, min_length=25)
summary_b_1= summary_b.replace('. ', '.\n')
print(f"Prompt: '{prompt_b}'")
print(f"Summary: {summary_b_1}")
```

```
 Prompt B: Brief Overview Request
Prompt: 'Provide a brief overview of the following text: '
Summary: The field of renewable energy has experienced unprecedented growth over the past decade.
Solar photovoltaic systems have become the cheapest source of electricity in many regions.
The International Energy Agency projects that renewable sources will account for 90% of the increase in  global electricity capacity through 2025.
```

```
# Prompt C: Key points extraction
print("\nPrompt C: Key Points Extraction ")
prompt_c = "Extract the main key points from this article: "
summary_c = generate_summary_with_prompt(test_article, prompt_c, max_length=80, min_length=25)
summary_c_1= summary_c.replace('. ', '.\n')
print(f"Prompt: '{prompt_c}'")
print(f"Summary: {summary_c_1}")
```

```
--- Prompt C: Key Points Extraction ---
Prompt: 'Extract the main key points from this article: '
Summary: The field of renewable energy has experienced unprecedented growth over the past decade.
Solar photovoltaic systems have become the cheapest source of electricity in many regions, with installation costs dropping by over 80% since 2010.
Wind energy has similarly benefited from larger, more efficient turbines.
```

```
# Prompt D: Executive summary style
print("\n Prompt D: Executive Summary Style")
prompt_d = "Write an executive summary of the following content: "
summary_d = generate_summary_with_prompt(test_article, prompt_d, max_length=80, min_length=25)
summary_d_1= summary_d.replace('. ', '.\n')
print(f"Prompt: '{prompt_d}'")
print(f"Summary: {summary_d_1}")
```

```
--- Prompt D: Executive Summary Style ---
Prompt: 'Write an executive summary of the following content: '
Summary: The field of renewable energy has experienced unprecedented growth over the past decade.
Solar photovoltaic systems have become the cheapest source of electricity in many regions.
The International Energy Agency projects that renewable sources will account for 90% of the increase in global electricity capacity through 2025.
```

OBSERVATIONS WITH PROMPT VARIATIONS:-

1. Standard prompt (no prefix): Generated general summary of the entire content
2. 'Brief overview' prompt: Produced a more concise, high-level summary
3. 'Key points' prompt: Focused on extracting specific important information
4. 'Executive summary' prompt: Created a more formal, business-oriented summary

Q5: Autoencoders – Representation Learning

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
import torch.nn.functional as F
```

```
# Hyperparameters
latent_dim_q5 = 32
batch_size_q5 = 128
epochs_q5 = 15     # Train for 15 epochs
learning_rate_q5 = 0.001
```

```
# Loading the MNIST dataset and normalizing it.
print("Loading and normalizing MNIST dataset...")
transform_q5 = transforms.Compose([
    transforms.ToTensor(),
])
```

```
Loading and normalizing MNIST dataset...
```

```
train_dataset_q5 = datasets.MNIST(root='./data_mnist_cae', train=True, transform=transform_q5, download=True)
# Using train_loader for training and also for visualization samples
train_loader_q5 = DataLoader(dataset=train_dataset_q5, batch_size=batch_size_q5, shuffle=True)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 38.6MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.13MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 10.3MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 1.69MB/s]
```

```
test_dataset_q5 = datasets.MNIST(root='./data_mnist_cae', train=False, transform=transform_q5, download=True)
test_loader_q5 = DataLoader(dataset=test_dataset_q5, batch_size=batch_size_q5, shuffle=False)
print("MNIST dataset loaded.")
```

```
MNIST dataset loaded.
```

```
# Building a simple convolutional autoencoder:Encoder (Conv + MaxPooling), Decoder (UpSampling + Conv)
class ConvAutoencoderQ5(nn.Module):
    def __init__(self, latent_dim=latent_dim_q5):
        super(ConvAutoencoderQ5, self).__init__()
        # Encoder
        # Input:
        self.encoder_conv = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # Flatten and passing through a Linear layer
        self.encoder_fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, latent_dim),
            nn.ReLU()
        )

        #  Decoder
        self.decoder_fc = nn.Sequential(
            nn.Linear(latent_dim, 32 * 7 * 7),
            nn.ReLU()
        )
        self.decoder_unflatten = nn.Unflatten(1, (32, 7, 7))

        self.decoder_conv = nn.Sequential(
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.Sigmoid() # To ensure output pixel values are between 0 and 1
        )
    def forward(self, x):
        # Encode
        x_conv_out = self.encoder_conv(x)
        bottleneck_features = self.encoder_fc(x_conv_out)

        # Decode
        x_fc_decode_out = self.decoder_fc(bottleneck_features)
        x_unflattened = self.decoder_unflatten(x_fc_decode_out)
        reconstructed_x = self.decoder_conv(x_unflattened)

        return reconstructed_x, bottleneck_features
```

```
model_cae_q5 = ConvAutoencoderQ5(latent_dim=latent_dim_q5).to(device)
print("\nConvolutional Autoencoder model structure:")
print(model_cae_q5)
```

```
Convolutional Autoencoder model structure:
ConvAutoencoderQ5(
  (encoder_conv): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (encoder_fc): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=32, bias=True)
    (2): ReLU()
  )
  (decoder_fc): Sequential(
    (0): Linear(in_features=32, out_features=1568, bias=True)
    (1): ReLU()
  )
  (decoder_unflatten): Unflatten(dim=1, unflattened_size=(32, 7, 7))
  (decoder_conv): Sequential(
    (0): ConvTranspose2d(32, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (1): ReLU()
    (2): ConvTranspose2d(16, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (3): Sigmoid()
  )
)
```

```
def train_cae_q5(model, dataloader, epochs, lr):
    model.train() # Set model to training mode
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.MSELoss() # Mean Squared Error for reconstruction

    print(f"\nStarting CAE training for {epochs} epochs...")
    for epoch in range(epochs):
        total_epoch_loss = 0
        for batch_images, _ in dataloader:
            batch_images = batch_images.to(device)

            optimizer.zero_grad()
            reconstructed_images, _ = model(batch_images)
            loss = criterion(reconstructed_images, batch_images)
            loss.backward()
            optimizer.step()

            total_epoch_loss += loss.item() * batch_images.size(0)

        avg_epoch_loss = total_epoch_loss / len(dataloader.dataset)
        print(f"Epoch [{epoch+1}/{epochs}], Average Loss: {avg_epoch_loss:.6f}")
    print("CAE Training complete.")
```

```
train_cae_q5(model_cae_q5, train_loader_q5, epochs=epochs_q5, lr=learning_rate_q5)
```

```
Starting CAE training for 15 epochs...
Epoch [1/15], Average Loss: 0.055522
Epoch [2/15], Average Loss: 0.020369
Epoch [3/15], Average Loss: 0.016906
Epoch [4/15], Average Loss: 0.015242
Epoch [5/15], Average Loss: 0.014157
Epoch [6/15], Average Loss: 0.013401
Epoch [7/15], Average Loss: 0.012859
Epoch [8/15], Average Loss: 0.012412
Epoch [9/15], Average Loss: 0.012064
Epoch [10/15], Average Loss: 0.011767
Epoch [11/15], Average Loss: 0.011524
Epoch [12/15], Average Loss: 0.011314
Epoch [13/15], Average Loss: 0.011114
Epoch [14/15], Average Loss: 0.010951
Epoch [15/15], Average Loss: 0.010810
CAE Training complete.
```

```
print("\n Results :- ")
model_cae_q5.eval() # Set model to evaluation mode
```

```
 Results :-
ConvAutoencoderQ5(
  (encoder_conv): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (encoder_fc): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=32, bias=True)
    (2): ReLU()
  )
  (decoder_fc): Sequential(
    (0): Linear(in_features=32, out_features=1568, bias=True)
    (1): ReLU()
  )
  (decoder_unflatten): Unflatten(dim=1, unflattened_size=(32, 7, 7))
  (decoder_conv): Sequential(
    (0): ConvTranspose2d(32, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (1): ReLU()
    (2): ConvTranspose2d(16, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (3): Sigmoid()
  )
)
```

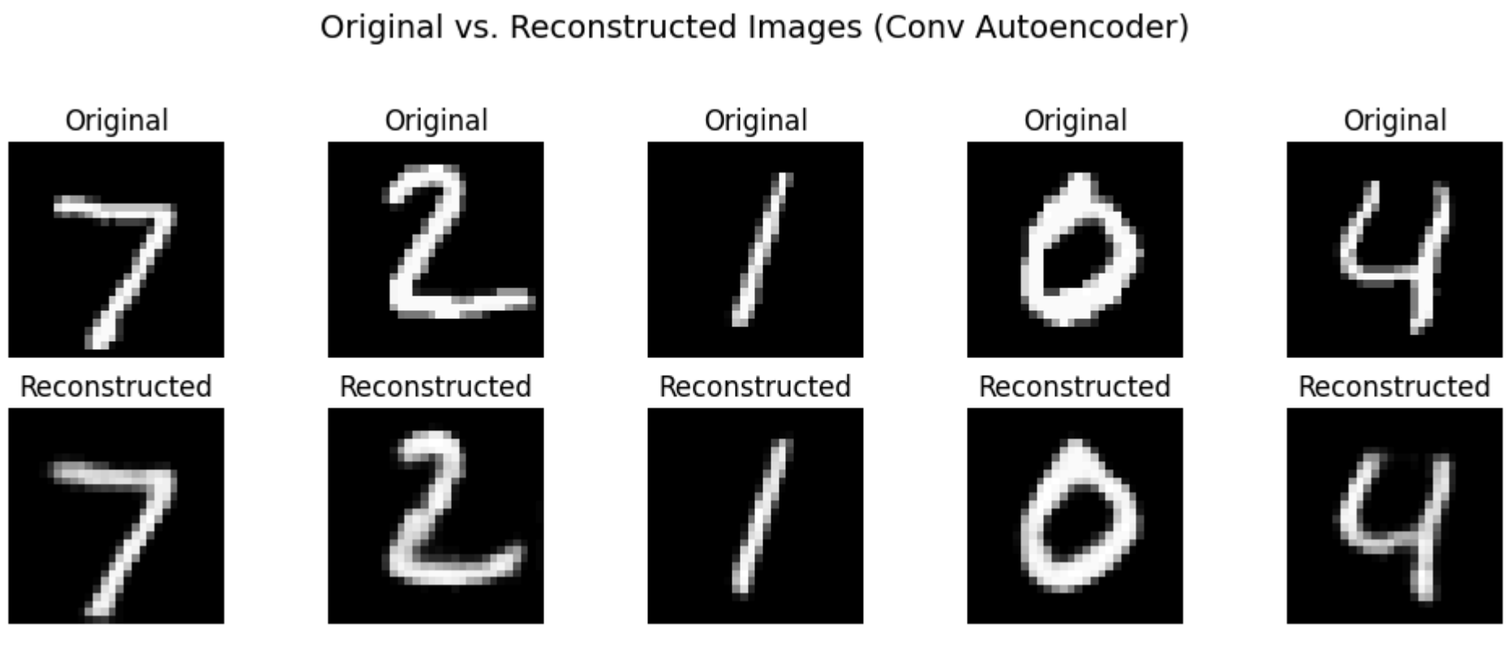
```
# Get a batch of images from the test_loader for visualization and bottleneck shape
sample_data_iter = iter(test_loader_q5)
sample_images_q5, _ = next(sample_data_iter)
sample_images_q5 = sample_images_q5.to(device)
```

```
with torch.no_grad():
    reconstructed_imgs_q5, bottleneck_out_q5 = model_cae_q5(sample_images_q5)
```

```
# Bottleneck layer shape
print(f"\nII) Bottleneck layer output shape for a sample batch: {bottleneck_out_q5.shape}")
print(f"   (Batch Size, Latent Dimension = {latent_dim_q5})")
```

```
II) Bottleneck layer output shape for a sample batch: torch.Size([128, 32])
   (Batch Size, Latent Dimension = 32)
```

```
# Original vs Reconstructed images
print("\nI) Displaying Original vs. Reconstructed Images...")
sample_images_np_q5 = sample_images_q5.cpu().numpy()
reconstructed_imgs_np_q5 = reconstructed_imgs_q5.cpu().numpy()
```

```
I) Displaying Original vs. Reconstructed Images...
```

```
n_display_q5 = 5
plt.figure(figsize=(10, 4))
plt.suptitle("Original vs. Reconstructed Images (Conv Autoencoder)", fontsize=14)
for i in range(n_display_q5):
    # Displaying the  original images
    ax = plt.subplot(2, n_display_q5, i + 1)
    plt.imshow(sample_images_np_q5[i].reshape(28, 28), cmap='gray')
    ax.set_title("Original")
    ax.axis('off')

    # Displaying the reconstructed images
    ax = plt.subplot(2, n_display_q5, i + 1 + n_display_q5)
    plt.imshow(reconstructed_imgs_np_q5[i].reshape(28, 28), cmap='gray')
    ax.set_title("Reconstructed")
    ax.axis('off')
plt.tight_layout(rect=[0, 0, 1, 0.95]) # Adjusting the layout
plt.show()
```